PORTFOLIO

이희준 포트폴리오

Github: **github.com/Spinichi**

Phone: **010-8851-7943**

Blog: **velog.io/@spinichi**



안녕하세요.
업무 흐름을 이해하고,
시스템의 정확성과 안정성을 개선하는
개발자 이희준입니다.

Java와 Spring 기반 API 개발부터 React 화면 구현,
배포 환경 구성까지 서비스 개발 전반을 경험했습니다.
기능별 데이터 흐름과 처리 조건을 살피고,
테스트와 로그 분석으로 데이터 정합성과 성능을 개선해 왔습니다.

ESB와 API Gateway 구축, 검증 및 기술지원 업무에서는
기존 시스템의 데이터 전달 조건과 처리 흐름을 분석하고,
데이터 연계 과정에서 발생하는 문제의 원인을 추적하고 있습니다.

경력

2026.04 ~ 현재	**인젠트 | 연계2팀** 시스템 연계(ESB) 및 API Gateway 구축, 검증 및 기술지원

학력 및 교육

2025.01 ~ 2025.12	삼성 청년 SW · AI 아카데미 수료 (SSAFY)
2023.02	인하대학교 신소재공학과 졸업

수상

2025.11	**SSAFY 프로젝트 우수상 수상** (HAND - 스마트 워치 기반 정신 건강 관리 서비스)
2025.09	**SSAFY 프로젝트 우수상 수상** (Udong - 동아리 관리 플랫폼)
2025.08	**SSAFY 프로젝트 우수상 수상** (AI Contact - AI 기반 커플 소통 플랫폼)

자격증

2025.06	정보처리기사 취득
2025.04	SQLD 취득
2024.03	ADsP 취득

기술 스택

Backend

Java & Spring Boot
- 3개 프로젝트의 **REST API 설계 및 구현**
- **JPA와 Repository 인터페이스** 활용한 데이터 조회 및 필터링

Spring Security, JWT
- **사용자 인증 및 권한 관리 시스템** 구현
- Stateless 토큰 기반 인증 처리

Frontend

React, TypeScript, Tailwind CSS
- **Udong 프로젝트 UI/UX 개발**
- **공통 UI 컴포넌트 구현 및 재사용**
- 일관된 디자인 시스템 적용

Zustand
- 전역 상태 관리 (인증, 동아리 정보 등)

Infrastructure & DevOps

AWS (EC2, RDS, S3)
- EC2 4대로 AI/Backend/DB/Nginx 서버 분리 구축 (HAND)
- **RDS(MySQL) DB 서버 운영** (AI Contact)
- **S3를 활용한 이미지/영상 저장** (AI Contact)

Jenkins
- GitLab Webhook 연동 CI/CD 파이프라인 구축
- 모노레포 환경에서 변경 감지 및 **선택적 배포로 배포 시간 최대 50% 단축**

Database

MySQL
- **프로젝트의 테이블, 관계 설계 및 SQL 조회 구현**
- **실행계획 분석과 복합 인덱스 적용으로 조회 성능 개선**
- 트랜잭션과 잠금을 활용한 동시 요청의 중복 처리 방지

Oracle
- **인터페이스 명세 기반 테이블 생성 및 테스트 SQL 작성**
- 고객사 환경의 DB 연동 및 초기 데이터 적재 오류 대응

Tools & Collaboration

Git, GitLab, Jira
- **브랜치 전략 수립 및 버전 관리**
- 스프린트 기반 일정 관리 및 이슈 트래킹

PROJECTS

01 HAND (Have A Nice Day) - 스마트 워치 기반 정신 건강 관리 서비스

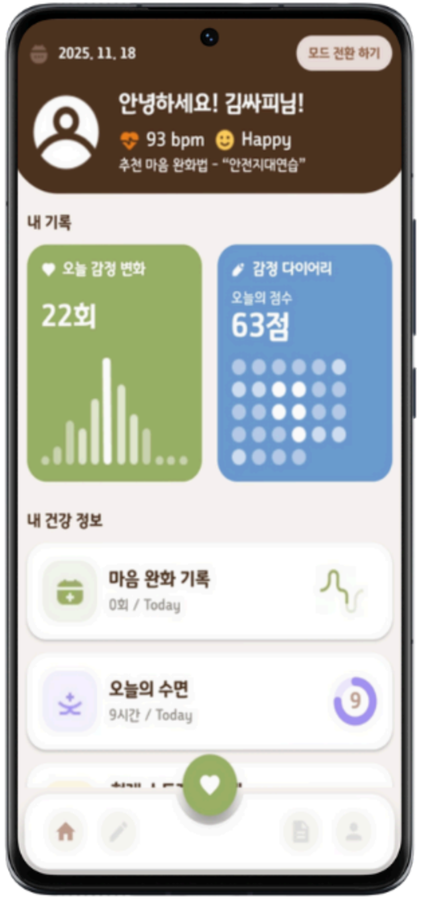
02 Udong - 동아리 관리 플랫폼

03 AI Contact - AI 기반 커플 소통 플랫폼

01. HAND (Have A Nice Day)

스마트 워치의 생체 데이터를 실시간 수집하여
AI 기반 감정 분석 및 맞춤형 심리 완화 조언을 제공하는 서비스

기간: 2025.10 ~ 2025.11 / 인원: 7명 / **역할: 팀장, 백엔드 개발 및 인프라**
성과: SSAFY 프로젝트 우수상 수상







기술스택

- **Backend: Spring Boot, JPA**
- **Database: MySQL**
- Infra: AWS EC2, Docker, Jenkins, Nginx
- Monitoring: Grafana, Prometheus, Loki
- App: Android(Kotlin)

구현 내용

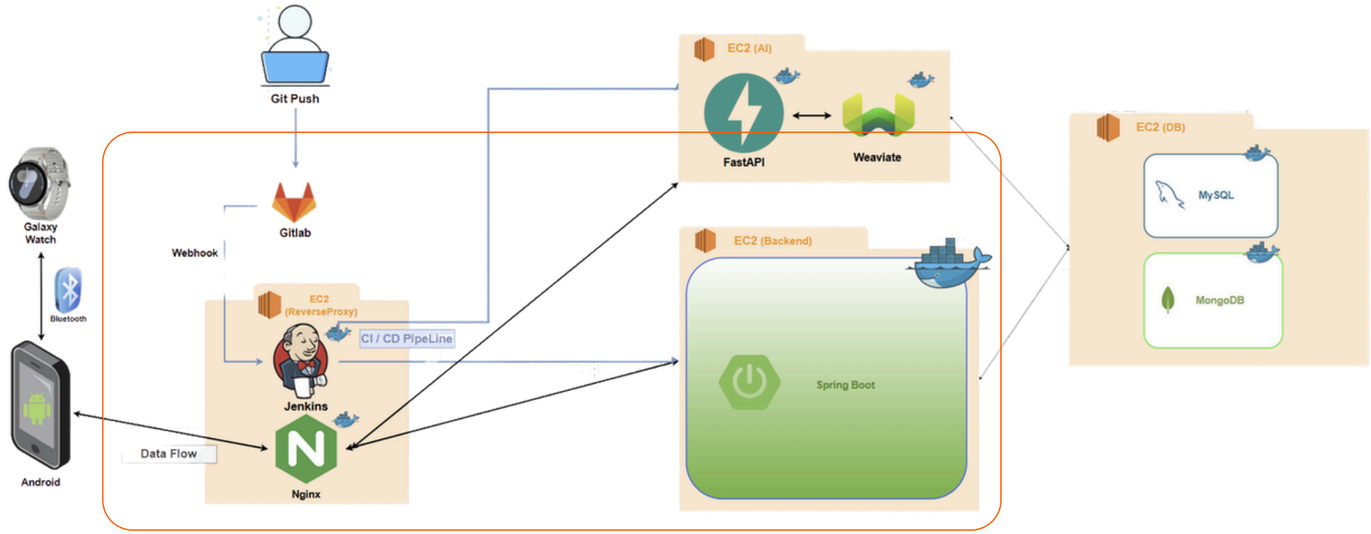
- **생체 데이터 수신과 저장, 사용자별 및 시간대별 조회 API 구현**
- **다이어리 완료 API 리팩토링**: OIV 비활성화 및 서비스 레이어 분리로 트랜잭션 경계 재정의, 외부 API 호출 구간 분리
- **주간 보고서 배치 리팩토링**: 동기 처리 구조를 비동기 병렬 처리로 전환하고, 청크 단위 백프레셔 관리로 처리 흐름 제어
- **요청 추적 로깅 개선**: JSONL 구조화 로깅 포맷 적용 및 MDC Trace ID로 요청 단위 로그 상관관계 구성
- CI/CD 자동화: Jenkinsfile 기반 빌드 → Docker 이미지 빌드/태깅 → EC2 배포 파이프라인 구성, Credentials로 시크릿 주입
- 운영 트래픽 라우팅 구성: Nginx 리버스 프록시(upstream/timeout/healthcheck) 설정으로 서비스 분리 환경 트래픽 라우팅 정리

핵심 기여

- **백엔드 성능/안정성 개선**
  - **OIV 비활성화 및 트랜잭션 경계 재정의**
  - **배치 비동기 병렬+동시 실행 상한**
  - **JSONL+TraceId 로깅**
- 운영 자동화/운영성
  - Jenkinsfile 기반 배포 자동화(시크릿 주입)
  - Nginx 리버스 프록시로 트래픽 라우팅 구성

아키텍처 설명

- 스마트 워치에서 수집한 생체 데이터를 앱이 전처리 후 백엔드 API로 전송
- 백엔드는 DB 저장/조회 및 AI 처리 연동을 담당하는 흐름
- AI/Backend/DB/Nginx를 분리 배치해 역할 경계와 장애 격리 확보



주요 담당 : 인프라 구축, 백엔드 성능 개선

문제 해결

1) 외부 API 지연 + OIV로 인한 커넥션 고갈 리스크 해결

문제

- 다이어리 완료 API에서 외부 API 지연이 발생할 때,
  OIV(Open-In-View)로 트랜잭션 경계가 과대하게 유지되는 패턴 확인
- 요청이 몰리면 DB 커넥션 점유 시간이 길어져 **커넥션 풀 고갈 및 응답 지연** 확대 가능성 발견

해결

- **외부 API 호출 구간을 서비스 클래스로 분리**하여,
  DB 트랜잭션과 결합되지 않도록 경계 재정의
- **OIV 비활성화**로 영속성 컨텍스트가 요청 전체를 감싸지 않도록 변경,
  DB 점유 시간을 최소화

결과

- **p95 응답시간 4,730ms → 3,086ms로 개선**
- **처리량 2.07 → 2.31 req/s로 증가**

2) 주간 보고서 배치 동기 병목 해결(비동기 병렬 + 백프레셔 관리)

문제

- 주간 보고서 배치가 **동기 처리 구조로 동작**해 **처리시간이 누적**되고,
  사용자 수 증가 시 확장성 한계 확인
- **스레드풀 포화**로 RejectedExecutionException 발생 가능성이 있어
  배치 안정성 저하 우려

해결

- **ThreadPoolTaskExecutor + @Async로 비동기 병렬 처리로 전환,
  CompletableFuture로 결과 수집**
- 스레드풀 포화 상황에서도 시스템이 무너지지 않도록
  chunkSize를 고정하여 동시 실행 상한을 유지하며 과부하를 제어

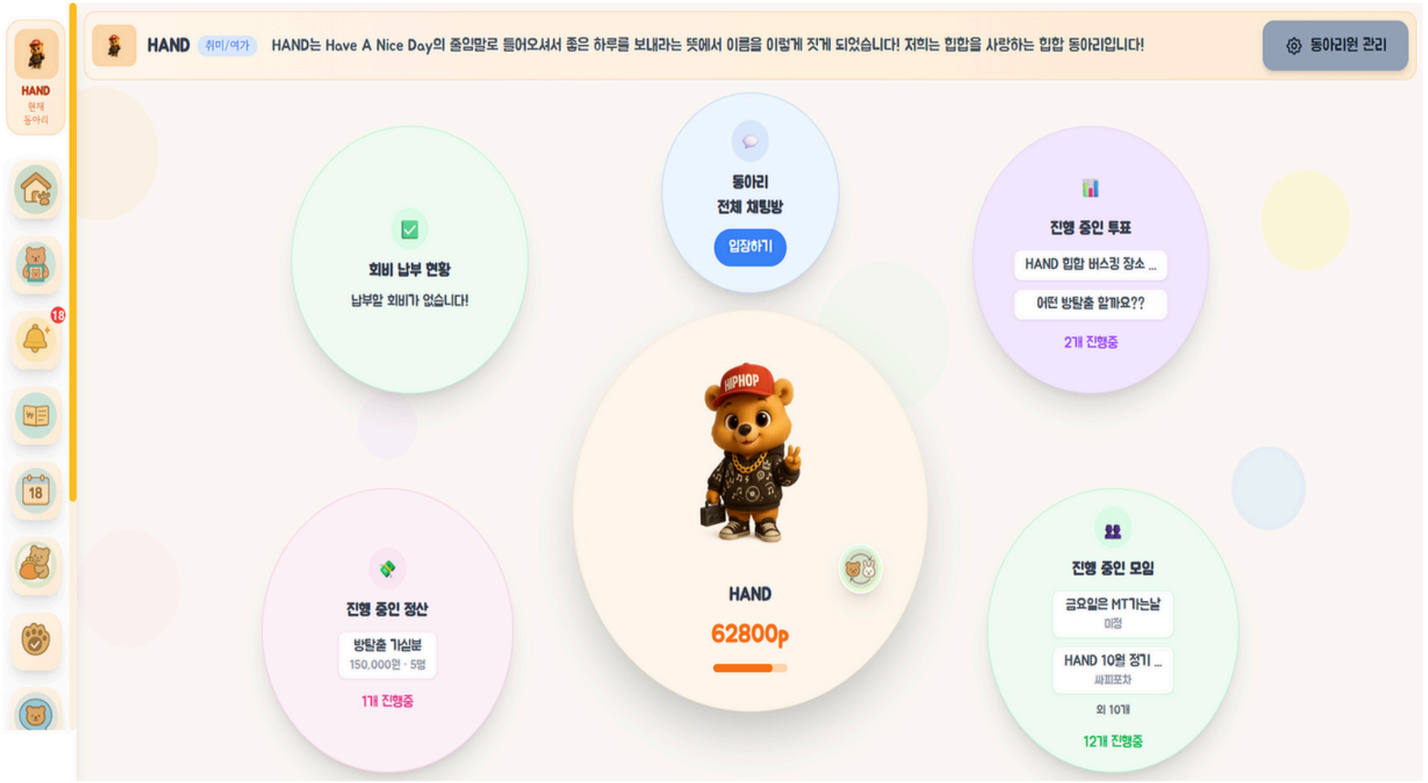
결과

- **처리시간 40.7s → 13.9s로 단축(유저 1,000명 기준)**
- **RejectedExecutionException 해소로 안정성 확보**

02. Udong (우리 동아리)

동아리 운영진의 회비 관리, 정산, 회원 관리의 복잡함을 해결하는 올인원 플랫폼

기간: 2025.08 ~ 2025.09 / 인원: 6명 / **역할: 팀장, 풀스택 개발**
성과: SSAFY 프로젝트 우수상 수상



기술스택

- **Backend: Spring Boot, JPA, WebSocket**
- **Database: MySQL**
- Frontend: React, TypeScript, Tailwind CSS
- Tools: Jira, GitLab, Notion

구현 내용

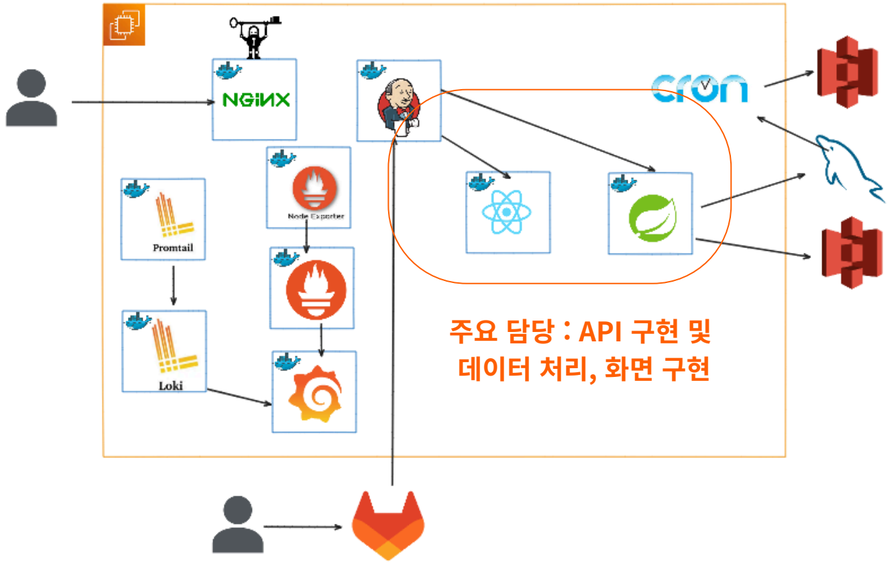
- **회원 관리 API: 등급/회비 납부 상태 등 조건별 필터링 가능한 회원 관리 조회 API 구현**
- **프론트엔드 구현: React/TypeScript 기반 기능 구현 및 공통 UI 컴포넌트/레이아웃 구축으로 화면 흐름과 UI 일관성 정리**
- **동시성 제어: 투표 참여 중복 요청 구간에 JPA 비관적 락(Pessimistic Lock) 적용으로 중복 데이터 방지**
- 쿼리/인덱스 최적화: 투표 참여 여부 조회 쿼리에 Covering Index(복합 인덱스) 설계 및 적용
- 캐시 도입 검증: 미읽음 알림 개수 조회에 Redis 캐시 적용을 검토하고, k6 부하 테스트 기반으로 도입 여부 판단

핵심 기여

- **백엔드 성능/정확성 개선**
  - **Redis 캐시 도입 적합성 검증(k6 부하 테스트 기반 의사결정)**
  - **비관적 락(Pessimistic Lock) 기반 중복 요청 동시성 제어**
  - **Covering Index(복합 인덱스) 적용으로 조회 쿼리 최적화**
- 백엔드 품질/운영성
  - GlobalExceptionHandler 기반 결제 예외 처리 표준화

아키텍처 설명

- React 기반 SPA에서 REST API로 주요 기능을 호출하고,
  실시간 이벤트/상태 동기화는 WebSocket으로 처리하는 구조
- Spring Boot가 인증·비즈니스 로직·데이터 접근을 담당하며
  API/실시간 채널을 단일 백엔드에서 제공하는 흐름



문제 해결

1) 투표 참여 중복 요청 Race Condition 해결 — 비관적 락 적용

문제

- 투표 참여 API에서 existsByVoteIdAndUserId() → saveAll()로 이어지는 **CHECK-THEN-ACT 패턴 사용**
- 동시 요청(멀티탭/재시도/더블클릭) 구간에서 **트랜잭션 간 끼어들기 발생** →
  JMeter 200 동시 요청 조건에서 **중복 15건(7.5%) 발생**.

해결

- 투표 조회 시점에 **@Lock(PESSIMISTIC_WRITE) 적용** →
  SELECT ... FOR **UPDATE 기반 행 잠금** 획득.
- findByIdWithOptionsForUpdate()로 **락 보유 상태에서**
  중복 체크/참여 처리 수행 → 원자성 보장

결과 (JMeter 200회 요청 기준)

- **중복 발생 15건(7.5%) → 0건**
- **평균 응답시간 2837ms → 2039ms (약 28% 개선)**

2) 투표 참여 여부 조회 쿼리 최적화 — Covering Index 적용

문제

- “WHERE vote_id = ? AND user_id = ?” 형태의 존재 여부 체크 쿼리 빈번 실행
- **단일 인덱스(vote_id)만 존재** →
  vote_id로 1,000건 스캔 후 user_id 필터링 패턴 확인
- **EXPLAIN ANALYZE 기준 실행시간 0.971ms 소요 (1,000 rows 스캔 기준)**

해결

- **(vote_id, user_id) 복합 인덱스 추가 → Covering Index lookup 유도**
- 테이블 접근 없이 인덱스만으로 결과 반환 → Filter 단계 제거

결과 (1,000 rows 기준)

- **실행시간 0.971ms → 0.032ms 개선**
- **실행 계획 Index lookup → Filter → Covering index lookup 전환**

03. AI Contact

커플이 AI 가상 아기를 함께 키우며 관계를 유지하고,
AI 아기가 중재자 역할을 수행하여 소통을 증진하는 서비스

기간: 2025.07 ~ 2025.08 / 인원: 6명 / **역할: 백엔드 개발 및 인프라**
성과: SSAFY 프로젝트 우수상 수상



기술스택

- **Backend: Spring Boot, Spring Security, JWT, JPA**
- **Database: MySQL(AWS RDS)**
- Infra: AWS (EC2, RDS, S3), Docker, Jenkins, Nginx
- Real-time: LiveKit (WebRTC), WebSocket
- AI API: OpenAI (GPT-4o, DALL-E 3), Google Imagen 3

구현 내용

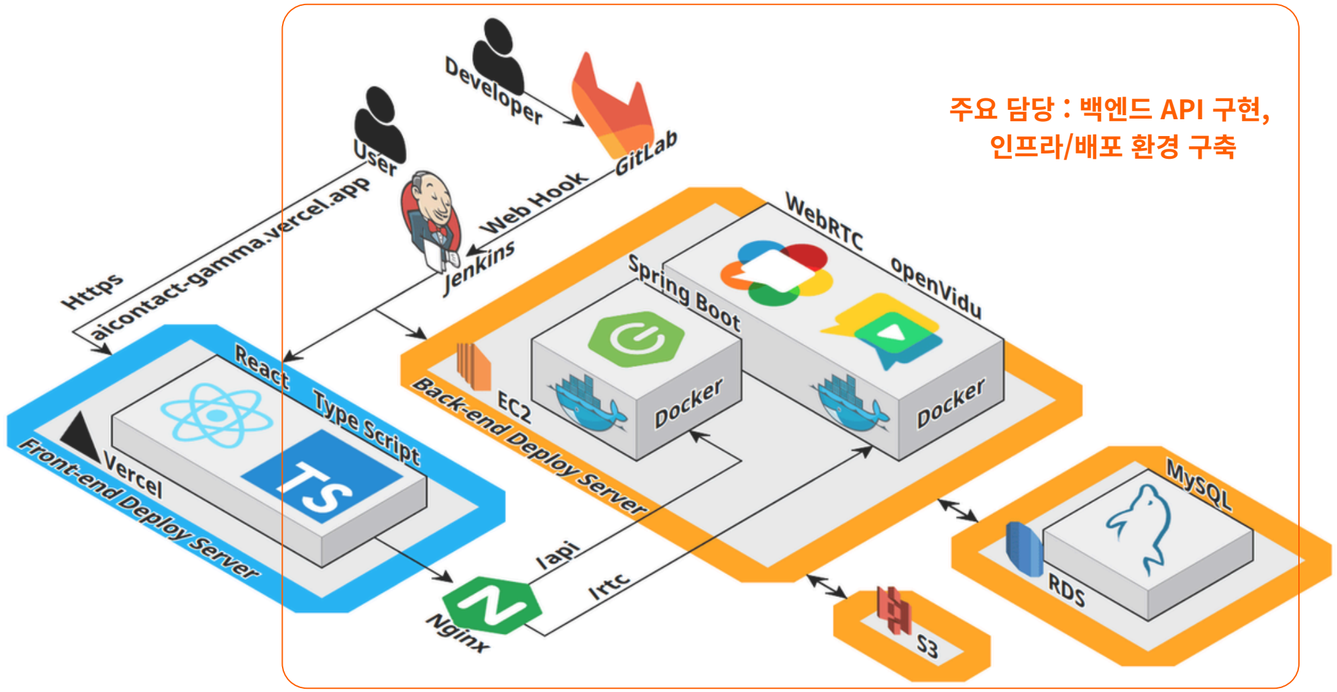
- **AI 기능 연동 백엔드 구현: GPT/DALL·E/Imagen 호출을 위한 API 클라이언트 및 요청/응답 처리 흐름을 구성해 이미지/콘텐츠 생성 기능을 서버에 통합**
- **JPA 조회 성능 개선:** 관리자 커플 목록 조회에서 발생한 N+1 문제를 인지하고**, Fetch Join 기반 조회 쿼리로 단일 쿼리 패턴으로 개선**
- **외부 API 안정성 강화: WebClient 타임아웃 설정과 Spring Retry(Exponential Backoff) 정책을 적용**해 외부 장애 전파를 완화
- **DB 커넥션 풀 튜닝: HikariCP Pool size를 부하 시나리오 기반으로 단계적 실험 및 튜닝하고, 운영 환경에 맞는 설정값을 도출**
- Docker 보안 강화: 컨테이너 런타임을 non-root(appuser)로 전환하고 최소 권한 원칙에 맞게 Dockerfile 실행 환경을 정리

핵심 기여

- **백엔드 성능/안정성 개선**
  - **N+1 제거(Fetch Join)**
  - **Timeout + Retry(Exponential Backoff)**
  - **HikariCP 튜닝(pool size 조정)**
- Jenkins CI/CD 파이프라인 구성 및 S3 업로드 연동으로 운영 흐름 정리

아키텍처 설명

- 백엔드와 LiveKit(WebRTC)을 Docker Compose로 함께 구동해 동일한 실행 환경에서 서비스가 동작하도록 구성
- 이미지/미디어는 AWS S3, DB는 AWS RDS를 활용하여 저장 및 조회 흐름을 분리해 서버는 API 처리에 집중



문제 해결

1) HikariCP 커넥션 풀 튜닝으로 자원 낭비 및 지연 개선

문제

- HikariCP 권장값 고정 적용에 따른 프로젝트 환경 대비 **과도한 동시 커넥션 설정** 가능성 존재
- 불필요한 커넥션 유지로 인한 **메모리, 스레드 자원 낭비 위험** 및 DB 경합에 따른 지연 변동성 확대 가능성 확인

해결

- 부하 시나리오 고정 후 **pool size의 단계적 변경** 및 **10회 이상의 반복 테스트 수행**
- 응답 지연(p95), **커넥션 상태 지표 기반**으로 운영 환경 적합 설정값 도출

결과

- **최적 Pool Size 도출로 메모리 낭비 방지(17 → 5) 및 응답 시간 19.66ms → 17.48ms 개선**

2) 외부 API 장애 전파를 Timeout + Retry로 완화

문제

- 외부 API 지연/오류 발생 시 **요청 스레드 장시간 점유 및 연쇄 실패 확대** 가능성 확인
- 일시적 장애(타임아웃/5xx) 상황에서 **사용자 체감 에러율 상승** 리스크 존재

해결

- **WebClient 명시적 timeout 적용**으로 무한 대기 차단
- **Spring Retry + Exponential Backoff 적용**으로 일시적 오류 구간 재시도 정책 구성

결과

- **외부 장애 5% 주입 조건에서 사용자 체감 에러율 10.12% → 0.26%로 개선**